Appendix E – Machine Learning Logistic Regression

Section Overview

Introduction 2
Process 2
Forward Propagation 2
Backward Propagation 2
Setting the New Adjustment and Weight Factors 2
Alpha – Learning Rate 2
Updating Weight & Adjustment According to Alpha and Gradient Descent 3
Process Conclusion 3
Sample Code 3
Conclusion 6
Works Cited 6

Introduction

This section acts as proof that the extensions are completely realistic. I learned the knowledge in this document in July 2020 from a Stanford University deep learning course on Coursera taught by Professor Andrew Ng. The theoretical mathematics relevant to logistic regression are shown below (Ng, 2017) in chronological order of calculation. Calculus proofs are not shown as this is a computer science internal assessment.

Process

Forward Propagation

Process	**Explanation**
$z = w^T x + b$ $= w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$	x: features (what to factor in the decision making, e.g., how to determine if a country is good to travel to? Covid has a factor? Living and travel expenses? Activity cost? Hotel price?). $w^T x$: transformed matrix of weights multiplied by the feature values.
$\hat{y} = a = \sigma(z)$	Final predictions. $\sigma(z)$: sigmoid function.
$L(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$	Loss function to minimize. a: logistic regression output. y: ground truth label.

Table AE.1: forward propagation process in chronological order. Recall the goal of forward propagation is to set values of "w" and "b" to minimize "L(a, y)". Note this is the computation for one training example.

Backward Propagation

$$dz = \frac{dL}{dz} = a - y$$
$$dw_1 = \frac{\partial L}{\partial w_1} = x_1 dz$$
$$dw_2 = \frac{\partial L}{\partial w_2} = x_2 dz$$
$$dw_n = \frac{\partial L}{\partial w_n} = x_n dz$$
$$db = dz$$

Backward propagation seeks to calculate the weights based on the loss function using gradient descent. When taking the derivative with respect to a variable, that variable is the same as the ones in Table A.E.1.

Setting the New Adjustment and Weight Factors

Alpha – Learning Rate

Note, alpha α is the learning rate. Gradient descent is the process of finding the local minimum of a loss function. The learning rate determines how big of a step to take in each attempt to approach a local minimum. Big learning rates may overshoot the local minimum. Small learning rates may take too long to reach a minimum. The overall problem with gradient descent is the insurmountable task of finding the local minimum because once the algorithm steps in one valley, it will approach the minimum point even if it is not the global minimum. See Figure A.E.1 and Figure A.E.2. How to set alpha? A simple way is to do trial and error to see which alpha minimizes the cost function.

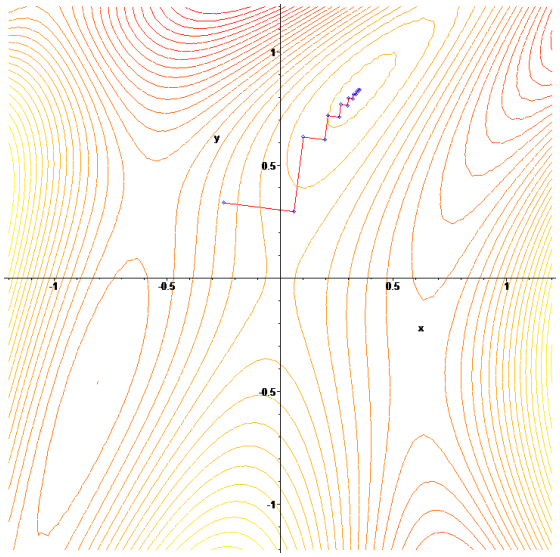

Figure AE.1: countour graph of the gradient descent showing the approximation to a local minimum (Gillis, *The gradient descent algorithm in action. (1: contour)* 2006).

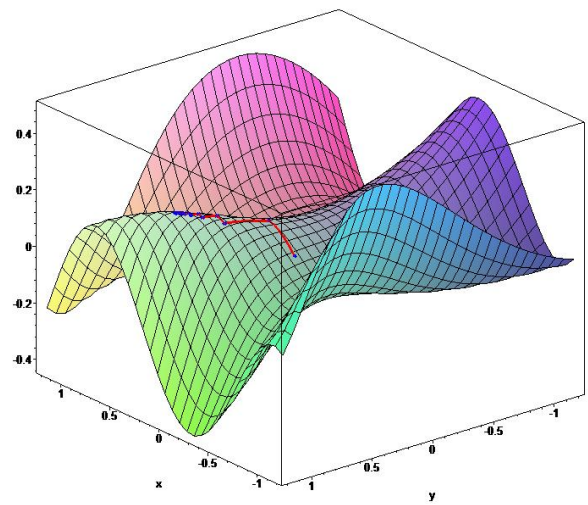

Figure AE.2: 3D model showing the approximation to a local minimum in a red zigzag line (Gillis, *The gradient descent algorithm in action. (2: surface)* 2006).

Updating Weight & Adjustment According to Alpha and Gradient Descent

$$w_1 = w_1 - \alpha\, dw_1$$
$$w_2 = w_2 - \alpha\, dw_2$$
$$w_n = w_n - \alpha\, dw_n$$
$$b = b - \alpha\, db$$

Process Conclusion

Perform forward and backwards propagation a few times (e.g., 100 each, alternating) to find the “w” and “b” on training data, then predict with new data.

Sample Code

The following code showcases a logistic regression algorithm to identify an ASCII circle. This is an exercise I created for myself for entertainment in July 2020. Note that for simplicity, a circle is defined by any enclosed space that looks somewhat like a circle. The ambiguity of the definition is to challenge the AI to finding the most suitable “w” and “b” for this task. The following code is a simple implementation of the theory described above. This code only need to be altered slightly to have the travel details as feature values. The training data and testing data would need alteration as well, but that is all that needs to be changed.

```
# GENERAL LOGIC
# INPUT=====
#       there are 16 test cases in each file (set). each test case starts with a number followed by 8 lines. the number represents y(i) which
 is the correct answer of the classification of the 8 lines that follow. the number is either 0 or 1. 0 represents there is a loop and 1 mean
s there isn't a loop. the next 8 lines are 8x8 matrix of an image where "*" represents a coloured pixel and everywhere else represents a non-
coloured pixel. the 8x8 matrix is inputed as a 1x64 matrix. all individual text cases (16 in total) combined together forms X which is a 16x6
4 matrix of all image input data. if in any 8x8 matrices there is a \n character, that represents the entire row should be filled with 0 (the
re isn't a "*") and input reads no longer than 8 characters on each of the 8x8 matrix image lines.
# FORWARD PROPOGATION=====
#       Z = w*X + b, where w is a 1x16 matrix of weights to be applied to each test case of X matrix (16x64) of images. b is a real number ad
ded to the result of X*w. Z is a 1x64 matrix of calculated data.
#       A=sigmoid(Z): also =y hat which is the calculated answer, sigmoid converts a value to a number from 0 to 1 exclusive.

import sys
import numpy as np
np.set_printoptions(threshold=sys.maxsize) # when output numpy array, output all items, without this line prints [a, b, ... , c, d] for examp
le
np.set_printoptions(suppress=True) # remove scientific notation when output numpy array

X, Y = [], [] # X: set of training data, in this example, 16 rows by 64 columns where each row is an individual test case (one image) each co
lumn represents whether a coordinate is coloured (denoted by "*" in input) or not from top left to bottom right of image pixels. Y: the class
```

```python
ification of whether the image is a loop or not 0 means is loop, 1 means not loop. 1 row and 16 columns in this example,represents the correc
t answer of the 16 test cases
w, b, Z, A, J = None, None, None, None, None # w: [1x64] matrix of all the individual weights to be applied to x(i) <-
- all 64 coordinates of a single image. b: a single real number added to the each result of w(i)*x(i), that is, each pixel is added with the
value "b" after calculating w*b. A: calculated result using forward propogation, a.k.a. "y hat", it is the output result [16x1] matrix that i
s compared to Y [16x1] matrix that was inputted. J: the cost function, which is the sum of all the losses, which is the sum of all the errors
, it shows the closeness of the calculated output compared to the desired output, the cost function J is wished to be minimized, the smaller
the better.
ALPHA = 0.106011575 # learning rate of backwards propogation, how big of a step to take in the gradient descent. the smaller, the smaller the
 step, however, a large alpha can skip the local minima. adjusting this number and the number of iterations of forward and backward propogati
on can produce optimal values for the current training set.

def input(dir): # take input by reading file given through "dir" e.g. dir = "test.txt"
    global X, Y # allows changes to X and Y be changed globally.
    inputX, inputY = [], [] # placeholders for the calculation of the X[] and Y[] matrices, since X and Y are numpy arrays while inputX and i
nputY are lists so if input multiple files, append() will not work with numpy array.
    tempX, setLine = [], -
1 # tempX: 1 row matrix for turning the 8 rows of input per image to a single line of 0 or 1. setLine: line number in current image, 8 lines
per image, -1 means current line is not an 8x8 image input. 1-8 inclusive represents the line number of the 8x8 image input.
    for line in open(dir, 'r').readlines(): # loop through file line by line
        if setLine != -1: # read the next if statement on the same level at line 23 first.
            tempX.extend(list("00000000") if(len(line.rstrip()) == 0) else list(line[:8].replace(" ", "0").replace("*", "1")))
            if setLine == 8:
                setLine = -1
                inputX.append(list(tempX)) if (len(tempX) != 0) else None
                tempX.clear()
            else:
                setLine += 1
        if line.rstrip().isnumeric(): # rstrip() removes trailing characters including whitespaces and excape characters. isnumeric() checks
if the input is a number.
            inputY.append(line.rstrip())
            setLine = 1
    X, Y = np.array(inputX, dtype = np.float64), np.array(inputY, dtype = np.float64)

def sigmoid(n):
    return 1/(1+np.exp(-n)) # exp: natural log (ln)

input("trainset1.txt")
w = np.full((1, len(X[0])), 1, dtype = np.float64)
b = 0.0

def forwardProp():
    global Z, A, J, w, b
    Z = np.dot(X, w.T) + b
    A = sigmoid(Z)
    J = -(np.dot(Y, A) + np.dot(np.subtract(1, Y), np.log(np.subtract(1, A))))

def backwardProp():
    global w, b
    dZ = np.subtract(A.T, Y)
    dw = np.dot(dZ, X)/len(X)
    db = np.sum(dZ)/len(X)
    w = np.subtract(w, dw*ALPHA)
    b = np.subtract(b, db*ALPHA)

forwardProp()
for i in range(100):
    backwardProp()
    forwardProp()

def output():
    print("======== RESULT ========")
    print("Cost (J): ", f"{J[0]:.11f}")
    for x in range(0, len(A)):
        for y in range(0, len(A[0])):
            print (np.round(A[x,y], 5), "\t", np.round(A[x,y], 1), "\t", Y[x])
    print()

output()

input("set0.txt")
forwardProp()
output()

print()
print()
print(Z)
```

Code AE.1: implementation of logistic regression with Python to identify an ASCII circle.

```
0
********
*      *
*      *
*      *
*      *
*      *
*      *
********
1
*      *
```

```
0
********
*      *
*      *
*      *
*      *
*      *
*      *
********
0
********
```

```
0
*******
*     *
*     *
*******
0
****
```

```
0
*******
*     *
*     *
*******
0
****
```

```
0
  * *
*     *
*     *
  * *
0
 **
```

```
*     *
*     *
*     *
*     *
*     *
*     *
********
1
********
      *
      *
      *
      *
      *
      *
********
1
********
*
*
*
*
*
*
********
1
********
*     *
*     *
*     *
*     *
*     *
*     *
*     *
1
********
********
1
*     *
*     *
*     *
*     *
*     *
*     *
*     *
*     *
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
```

```
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
0
********
*     *
*     *
```

```
*  *
*  *
****


0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
1
*  *
*  *
*  *
****



0
*******
*    *
*    *
*******



0



****
*  *
*  *
****
1


  * *
*    *
*    *

0
 ****
 *  *
 *  *
 ****


0
 *******
 *    *
 *    *
 *******


1

   **
  *
  *
   **


0


 *******
 *    *
 *    *
 *******

0

   ****
   *  *
   *  *
   ****

0

   ***
   * *
   ***
```

```
*  *
*  *
****


0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
1
*  *
*  *
*  *
****



0
*******
*    *
*    *
*******



0



****
*  *
*  *
****
1


  * *
*    *
*    *

0
 ****
 *  *
 *  *
 ****


0
 *******
 *    *
 *    *
 *******


1

   **
  *
  *
   **


0


 *******
 *    *
 *    *
 *******

0

   ****
   *  *
   *  *
   ****

0

   ***
   * *
   ***
```

```
*  *
*  *
 **


0
********
*     *
*     *
*     *
*     *
*     *
*     *
********
1
*  *
*  *
*  *
****



0
*******
*    *
*    *
*******



0

 **
*  *
*  *
 **
1


  * *
*    *
*    *

0
  **
 *  *
 *  *
  **


0
  * *
 *    *
 *    *
  * *


1

  **
 *
 *
  **


0


  *****
 *    *
 *    *
  *****

0

   **
  *  *
  *  *
   **

0

   *
  * *
   *
```

trainset1.txt	set0.txt	set1.txt	set2.txt	set3.txt
* * * * * * * * ******** 0 ******** * * * * * * * * * * * * ******** 0 ******** * * * * * * * * * * * * ******** 0 ******** * * * * * * * * * * * * ******** 0 ******** * * * * * * * * * * * * ********	* * * * * * * * ******** 0 ******** * * * * * * * * * * * * ******** 0 ******** * * * * * * * * * * * * ******** 0 ******** * * * * * * * * * * * * ******** 0 ******** * * * * * * * * * * * * ********	0 **** * * * * **** 1 1 *	0 **** * * * * **** 1 1 *	0 ** * * * * ** 1 1 *

Table AE.1: training and testing data. Explanation of these sets are in Code Snapshot A.E.1.

Conclusion

Implementation of this machine learning neural network logistic regression algorithm is very realistic in integrating it to the Coronavirus Analyzer program. This would help with mimicking the client's decision style, and thus make consistent decisions no matter the time of day. To connect this with the Coronavirus Analyzer Java program, it can be easily done by processing the data as well in Python, then exporting the results to a file, then have the java program read it. The process could be managed by a batch file.

Works Cited

Gillis, J. (2006, January 19). The gradient descent algorithm in action. (1: Contour). Retrieved April 03, 2021, from https://en.wikipedia.org/wiki/Gradient_descent#/media/File:Gradient_ascent_(contour).png

Gillis, J. (2006, January 19). The gradient descent algorithm in action. (2: Surface). Retrieved April 03, 2021, from https://en.wikipedia.org/wiki/Gradient_descent#/media/File:Gradient_ascent_(surface).png

Ng, A. (2017, August 25). Logistic regression gradient Descent (C1W2L09). Retrieved April 03, 2021, from https://www.youtube.com/watch?v=z_xiwjEdAC4